Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 5

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Введение в Kubernetes

Обучающийся Дощенников Никита Андреевич

Факультет Прикладной информатики

Группа К3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Дощенников Н.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург
2026

Содержание

1. Персональные параметры 3
2. Задание 1. Подготовка образа 4
3. Задание 2. Развертывание базы данных и кэша 4
4. Задание 3. Развертывание приложения 7
5. Задание 4. Проверка 9

1. Персональные параметры

Параметр	Значение
Номер ИСУ	*465797*
Персональный порт (app)	*5527*
Персональный порт (Redis)	*6467*
Персональный префикс	*doschennikov*
Секретный ключ	*secret97doschennikov*
Имя базы данных	*doschennikov97*

Таблица 1. Персональные параметры

2. Задание 1. Подготовка образа

Я загрузил образ напрямую в minikube:

```
minikube image load doschennikov-app:1.0
```

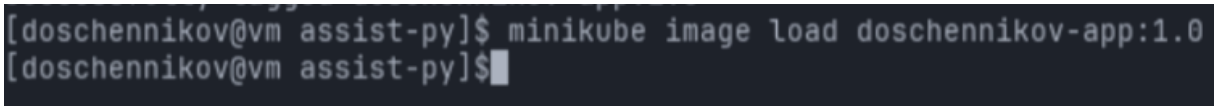


Рисунок 1. Загрузка образа в minikube

3. Задание 2. Развертывание базы данных и кэша

Я создал манифесты Kubernetes для PostgreSQL и Redis:

- `k8s/redis-service.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  type: ClusterIP
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
```

- `k8s/redis-deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      enableServiceLinks: false
      containers:
        - name: redis
          image: redis:7-alpine
```

```
          ports:
            - containerPort: 6379
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
            periodSeconds: 5
```

- `k8s/postgres-service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

- `k8s/postgres-pvc.yaml`:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
```

- `k8s/postgres-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
```

```
      labels:
        app: postgres
    spec:
      enableServiceLinks: false
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: POSTGRES_USER
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: POSTGRES_DB
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: POSTGRES_PASSWORD
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          readinessProbe:
            exec:
              command: ["sh", "-c", "pg_isready -U $POSTGRES_USER -d
$POSTGRES_DB"]
            initialDelaySeconds: 5
            periodSeconds: 5
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: postgres-pvc
```

- `k8s/app-secret.yaml`:

```
  apiVersion: v1
kind: Secret
metadata:
  name: app-secret
```

```
type: Opaque
stringData:
  APP_SECRET: "secret97doschennikov"
  POSTGRES_PASSWORD: "pgpass_465797"
```

- `k8s/app-configmap.yaml`:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_OWNER: "doschennikov"
  CACHE_TTL: "120"
  REDIS_URL: "redis://redis:6379/0"
  POSTGRES_HOST: "postgres"
  POSTGRES_PORT: "5432"
  POSTGRES_USER: "user_465797"
  POSTGRES_DB: "assistant_465797"
```

4. Задание 3. Развертывание приложения

Я создал манифесты для FastAPI-приложения:

- `k8s/app-deployment.yaml`:

```
  apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      enableServiceLinks: false
      initContainers:
        - name: wait-for-db
          image: postgres:15
          command: ["sh", "-c", "until pg_isready -h postgres -p
5432; do echo waiting for db; sleep 2; done"]
      containers:
```

```
        - name: app
          image: doschennikov-app:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5527
          envFrom:
            - configMapRef:
                name: app-config
            - secretRef:
                name: app-secret
          readinessProbe:
            httpGet:
              path: /health
              port: 5527
            initialDelaySeconds: 5
            periodSeconds: 5
```

- `k8s/app-service.yaml`:

```
  apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  type: NodePort
  selector:
    app: app
  ports:
    - port: 5527
      targetPort: 5527
      nodePort: 30527
```

5. Задание 4. Проверка

Я убедился, что приложение работает корректно. Для этого я посмотрел все ресурсы в неймспэйсе:

```
kubectl apply -f k8s/
kubectl get pods -w

kubectl get all
```

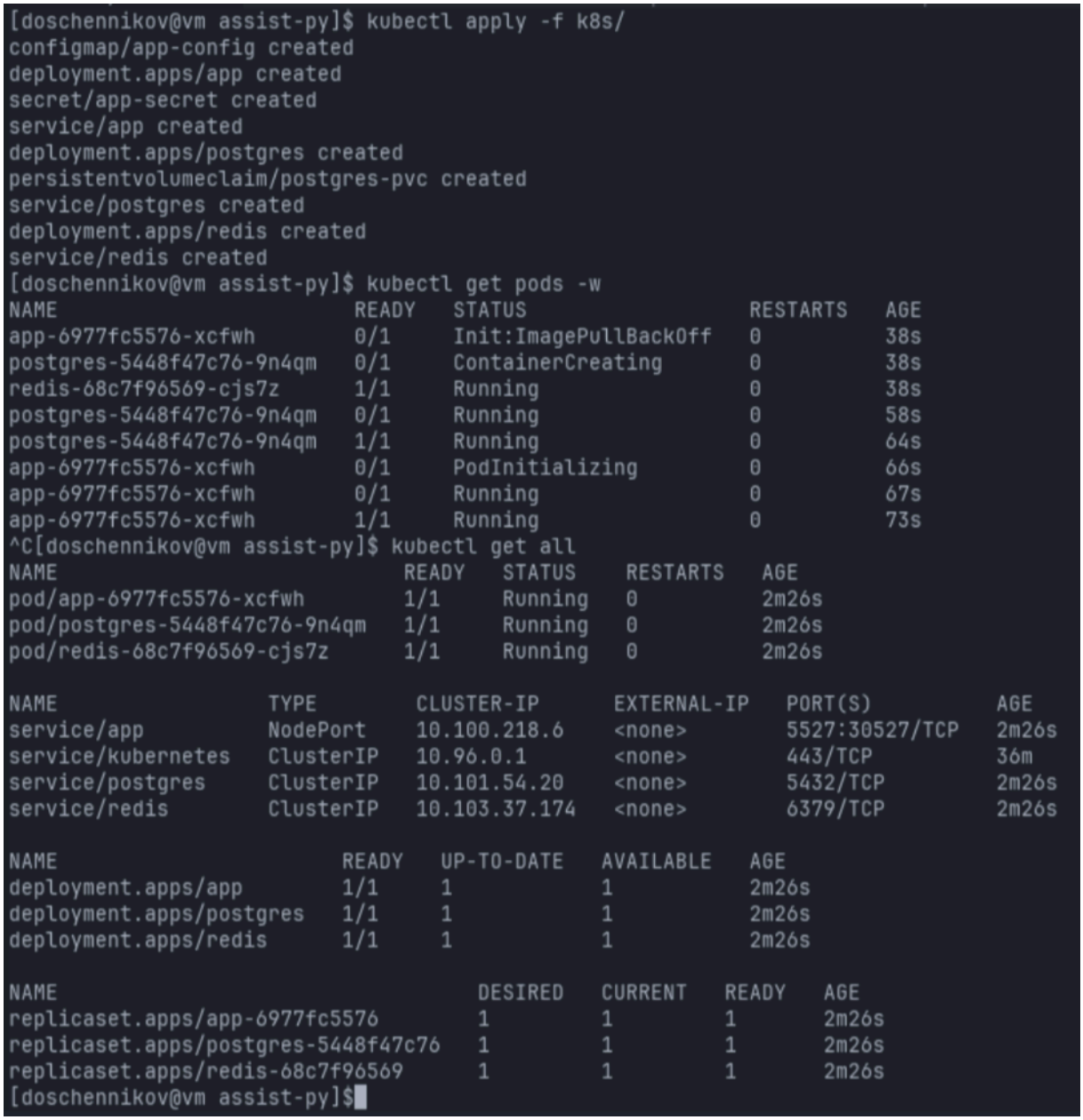


Рисунок 2. Ресурсы

Также я посмотрел логи пода приложения:

```
kubectl logs deploy/app
```

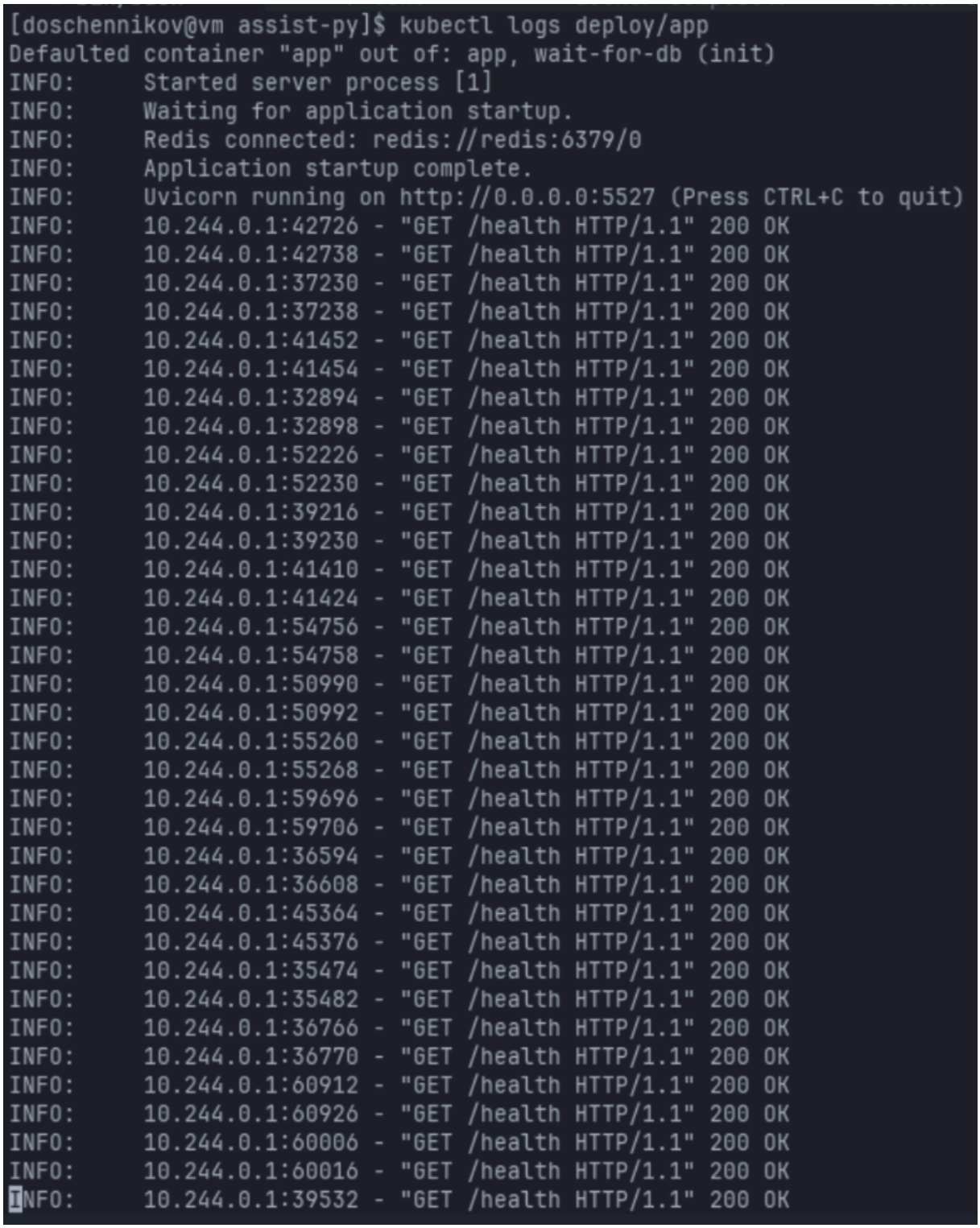


Рисунок 3. Логи пода приложения

Затем я открыл приложение:

```
minikube service app --url
```

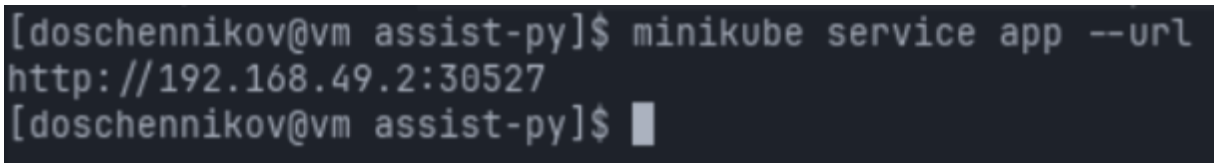


Рисунок 4. Адрес приложения

Затем я проверил все эндпоинты:

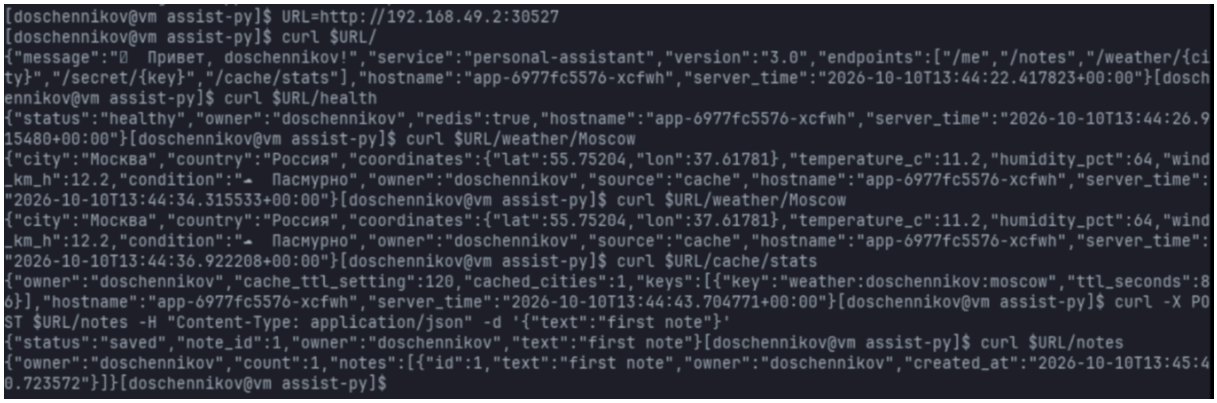


Рисунок 5. Проверка эндпоинтов